```
!pip install -q kaggle
```

```
from google.colab import files
files.upload()
```

Choose Files kaggle.json

kaggle.json(application/json) - 67 bytes, last modified: 2/25/2026 - 100% done

```
Saving kaggle.json to kaggle.json
{'kaggle.json': b'{"username":"annarose123","key":"09e4d9b618d9ddea1c963d4831693108"}'}
```

```
!unzip monet.zip
```

```
  inflating: trainB/2016-07-20 14_57_37.jpg
  inflating: trainB/2016-07-21 00_38_04.jpg
  inflating: trainB/2016-07-21 05_17_08.jpg
  inflating: trainB/2016-07-21 05_57_20.jpg
  inflating: trainB/2016-07-21 07_27_13.jpg
  inflating: trainB/2016-07-21 08_00_06.jpg
  inflating: trainB/2016-07-21 09_45_45.jpg
  inflating: trainB/2016-07-21 11_30_15.jpg
  inflating: trainB/2016-07-21 15_42_53.jpg
  inflating: trainB/2016-07-22 02_59_39.jpg
  inflating: trainB/2016-07-22 04_40_22.jpg
  inflating: trainB/2016-07-22 06_09_41.jpg
  inflating: trainB/2016-07-22 09_41_37.jpg
  inflating: trainB/2016-07-22 11_21_35.jpg
  inflating: trainB/2016-07-22 13_30_58.jpg
  inflating: trainB/2016-07-22 14_37_40.jpg
  inflating: trainB/2016-07-22 15_39_36.jpg
  inflating: trainB/2016-07-22 20_52_37.jpg
  inflating: trainB/2016-07-23 01_23_09.jpg
  inflating: trainB/2016-07-23 01_46_14.jpg
  inflating: trainB/2016-07-23 02_22_48.jpg
  inflating: trainB/2016-07-23 03_52_07.jpg
  inflating: trainB/2016-07-23 04_15_32.jpg
  inflating: trainB/2016-07-23 06_49_13.jpg
  inflating: trainB/2016-07-23 06_56_14.jpg
  inflating: trainB/2016-07-23 12_43_38.jpg
  inflating: trainB/2016-07-23 14_58_44.jpg
  inflating: trainB/2016-07-23 15_50_21.jpg
  inflating: trainB/2016-07-23 17_42_57.jpg
  inflating: trainB/2016-07-23 22_00_28.jpg
  inflating: trainB/2016-07-24 00_48_09.jpg
  inflating: trainB/2016-07-24 02_20_11.jpg
  inflating: trainB/2016-07-24 05_28_04.jpg
  inflating: trainB/2016-07-24 08_05_58.jpg
  inflating: trainB/2016-07-24 09_20_25.jpg
  inflating: trainB/2016-07-24 09_21_33.jpg
  inflating: trainB/2016-07-24 09_23_22.jpg
  inflating: trainB/2016-07-24 13_41_58.jpg
  inflating: trainB/2016-07-24 20_34_58.jpg
  inflating: trainB/2016-07-24 20_42_43.jpg
  inflating: trainB/2016-07-24 22_24_02.jpg
  inflating: trainB/2016-07-25 03_38_04.jpg
  inflating: trainB/2016-07-25 04_15_29.jpg
  inflating: trainB/2016-07-25 04_32_50.jpg
  inflating: trainB/2016-07-25 09_47_13.jpg
  inflating: trainB/2016-07-25 12_29_50.jpg
  inflating: trainB/2016-07-25 12_37_05.jpg
  inflating: trainB/2016-07-25 15_07_02.jpg
  inflating: trainB/2016-07-25 21_35_24.jpg
  inflating: trainB/2016-07-25 23_36_47.jpg
  inflating: trainB/2016-07-26 02_01_15.jpg
  inflating: trainB/2016-07-26 02_09_13.jpg
  inflating: trainB/2016-07-26 05_47_39.jpg
  inflating: trainB/2016-07-26 10_12_19.jpg
  inflating: trainB/2016-07-26 10_16_31.jpg
  inflating: trainB/2016-07-26 11_11_49.jpg
  inflating: trainB/2016-07-26 16_22_24.jpg
  inflating: trainB/2016-07-26 18_30_59.jpg
  inflating: trainB/2016-07-26 20_43_12.jpg
```

```
!ls
```

```
kaggle.json   monet.zip.zip                sample_data  testB   trainB
metadata.csv  pytorch-CycleGAN-and-pix2pix  testA        trainA
```

```
import torch
```

```
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
import os
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(device)
```

```
cpu
```

```
import os
from PIL import Image
from torch.utils.data import Dataset, DataLoader

class MonetDataset(Dataset):
    def __init__(self, root_dir, transform=None):
        self.root_dir = root_dir
        self.images = os.listdir(root_dir)
        self.transform = transform

    def __len__(self):
        return len(self.images)

    def __getitem__(self, idx):
        img_path = os.path.join(self.root_dir, self.images[idx])
        image = Image.open(img_path).convert("RGB")

        if self.transform:
            image = self.transform(image)

        return image

# Now use correct path
dataset = MonetDataset("/content/trainA", transform=transform)
dataloader = DataLoader(dataset, batch_size=32, shuffle=True)

print("Total images:", len(dataset))
```

```
Total images: 1072
```

```
epochs = 20

for epoch in range(epochs):
    for imgs in dataloader:
        real_imgs = imgs.to(device)
```

```
import torch
import torch.nn as nn

latent_dim = 100
img_size = 64   # keep 64 for CPU

class Generator(nn.Module):
    def __init__(self):
        super().__init__()
        self.model = nn.Sequential(
            nn.Linear(latent_dim, 256),
            nn.ReLU(),
            nn.Linear(256, 512),
            nn.ReLU(),
            nn.Linear(512, 3 * img_size * img_size),
            nn.Tanh()
        )

    def forward(self, z):
        img = self.model(z)
        return img.view(-1, 3, img_size, img_size)

G = Generator().to(device)
```

```
class Discriminator(nn.Module):
    def __init__(self):
```

```
            super().__init__()
            self.model = nn.Sequential(
                nn.Linear(3 * img_size * img_size, 512),
                nn.LeakyReLU(0.2),
                nn.Linear(512, 256),
                nn.LeakyReLU(0.2),
                nn.Linear(256, 1),
                nn.Sigmoid()
            )

        def forward(self, img):
            img_flat = img.view(img.size(0), -1)
            return self.model(img_flat)

    D = Discriminator().to(device)
```

```
    import torch.optim as optim

    optimizer_G = optim.Adam(G.parameters(), lr=0.0002)
    optimizer_D = optim.Adam(D.parameters(), lr=0.0002)
```

```
    criterion = nn.BCELoss()
```

```
    epochs = 20
    latent_dim = 100
```

```
    for epoch in range(epochs):
        for imgs in dataloader:

            real_imgs = imgs.to(device)
            batch_size = real_imgs.size(0)

            real_labels = torch.ones(batch_size, 1).to(device)
            fake_labels = torch.zeros(batch_size, 1).to(device)

            # =========================
            # Train Discriminator
            # =========================

            z = torch.randn(batch_size, latent_dim).to(device)
            fake_imgs = G(z)

            real_loss = criterion(D(real_imgs), real_labels)
            fake_loss = criterion(D(fake_imgs.detach()), fake_labels)

            d_loss = (real_loss + fake_loss) / 2

            optimizer_D.zero_grad()
            d_loss.backward()
            optimizer_D.step()

            # =========================
            # Train Generator
            # =========================

            z = torch.randn(batch_size, latent_dim).to(device)
            fake_imgs = G(z)

            g_loss = criterion(D(fake_imgs), real_labels)

            optimizer_G.zero_grad()
            g_loss.backward()
            optimizer_G.step()

        print(f"Epoch [{epoch+1}/{epochs}]  D_loss: {d_loss.item():.4f}  G_loss: {g_loss.item():.4f}")
```

```
Epoch [1/20]  D_loss: 0.1760  G_loss: 1.7642
Epoch [2/20]  D_loss: 0.0595  G_loss: 2.9758
Epoch [3/20]  D_loss: 0.0565  G_loss: 4.4153
Epoch [4/20]  D_loss: 0.0384  G_loss: 3.9277
Epoch [5/20]  D_loss: 0.0195  G_loss: 3.8882
Epoch [6/20]  D_loss: 0.0219  G_loss: 8.2078
Epoch [7/20]  D_loss: 0.0295  G_loss: 5.0529
Epoch [8/20]  D_loss: 0.1382  G_loss: 7.1527
Epoch [9/20]  D_loss: 0.0025  G_loss: 7.6150
```

```
Epoch [10/20]  D_loss: 0.0211  G_loss: 6.4281
Epoch [11/20]  D_loss: 0.0024  G_loss: 8.6114
Epoch [12/20]  D_loss: 0.0593  G_loss: 10.9745
Epoch [13/20]  D_loss: 0.0059  G_loss: 11.3939
Epoch [14/20]  D_loss: 0.0047  G_loss: 6.5655
Epoch [15/20]  D_loss: 0.0086  G_loss: 11.7669
Epoch [16/20]  D_loss: 0.0035  G_loss: 10.3620
Epoch [17/20]  D_loss: 0.0008  G_loss: 9.3679
Epoch [18/20]  D_loss: 0.0005  G_loss: 12.4099
Epoch [19/20]  D_loss: 0.0009  G_loss: 7.9725
Epoch [20/20]  D_loss: 0.0054  G_loss: 7.4135
```

```
from torchvision.utils import save_image

z = torch.randn(16, latent_dim).to(device)
generated_imgs = G(z)

save_image(generated_imgs, "vanilla_output.png", normalize=True)
```

```
import matplotlib.pyplot as plt
import torchvision

# Get real images
real_batch = next(iter(dataloader))
real_imgs = real_batch[:16]   # take 16 images
```

```
z = torch.randn(16, latent_dim).to(device)
fake_imgs = G(z).cpu()
```

```
real_grid = torchvision.utils.make_grid(real_imgs, nrow=4, normalize=True)
fake_grid = torchvision.utils.make_grid(fake_imgs, nrow=4, normalize=True)
```

```
plt.figure(figsize=(12,6))

# Real Images
plt.subplot(1,2,1)
plt.title("Original Monet Images")
plt.imshow(real_grid.permute(1,2,0))
plt.axis("off")

# Fake Images
plt.subplot(1,2,2)
plt.title("Vanilla GAN Generated Images")
plt.imshow(fake_grid.permute(1,2,0))
plt.axis("off")

plt.show()
```